CS-401 · PARALLEL AND DISTRIBUTED COMPUTING

Forms of Parallel Processing

Lecture 3 — Instruction & Data Streams, and Flynn's Taxonomy

Dr. Tariq Sadad

Department of Computer Science, IIUI

SETTING THE STAGE

Forms of Parallel Processing

Parallel computers are classified by the level at which hardware supports parallelism — and by the two streams that flow into every processor: instructions and data.

Hardware Level of Parallelism

Multi-core — multiple processing elements inside a single chip.

Multi-processor / clusters / grids — multiple whole computers cooperating on one task.

Two Streams Reach Every Processor

Instruction stream — the sequence of operations to perform.

Data stream — the values those operations act on.

This lecture covers parallelism from the instruction/data stream angle:

1 **Instruction-Level Parallelism (ILP)**

Reordering & overlapping instructions within one stream.

2 **Data-Level Parallelism (DLP)**

Spreading the same operation across many data items.

FORM 1

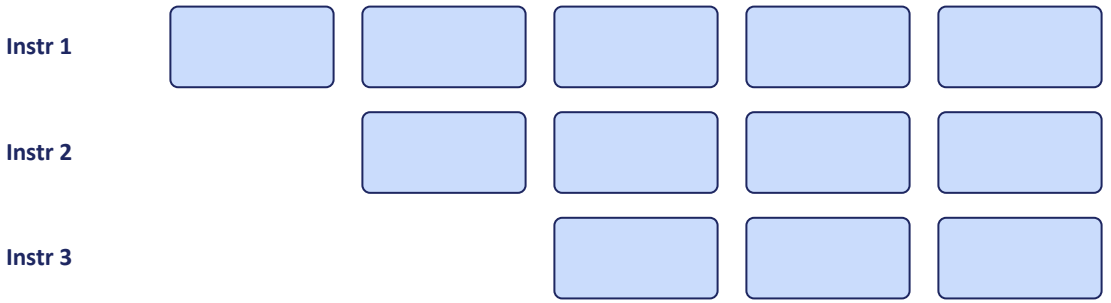
Instruction-Level Parallelism (ILP)

A program is a stream of instructions. ILP re-orders and groups independent instructions so they execute in parallel — without changing the program's result. Dominant approach from the mid-1980s to mid-1990s.

Example Classic RISC pipeline: 5 stages —
fetch → decode → execute → memory access → write-back.
The Pentium 4 pushed this to a 35-stage pipeline for higher clock speed.



Instruction pipeline — one instruction per clock cycle once full



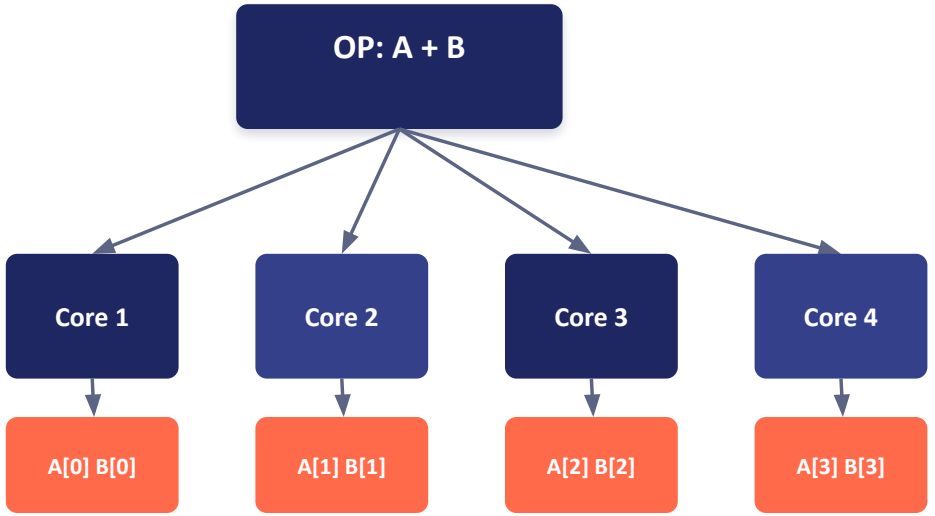
Overlap across cycles — three instructions in flight at once

FORM 2

Data-Level Parallelism (DLP)

The data stream is the sequence of inputs, and partial or temporary results, consumed by the instruction stream. DLP distributes data across computing nodes so the same calculation runs on many data items at once.

Example Adding two large arrays, A + B = C.
Split the arrays into 4 chunks and give one chunk to each core — every core runs the identical add operation on its own slice, simultaneously.
Used heavily in image filtering, weather simulation, and GPU shading.



Same instruction stream, four independent data streams

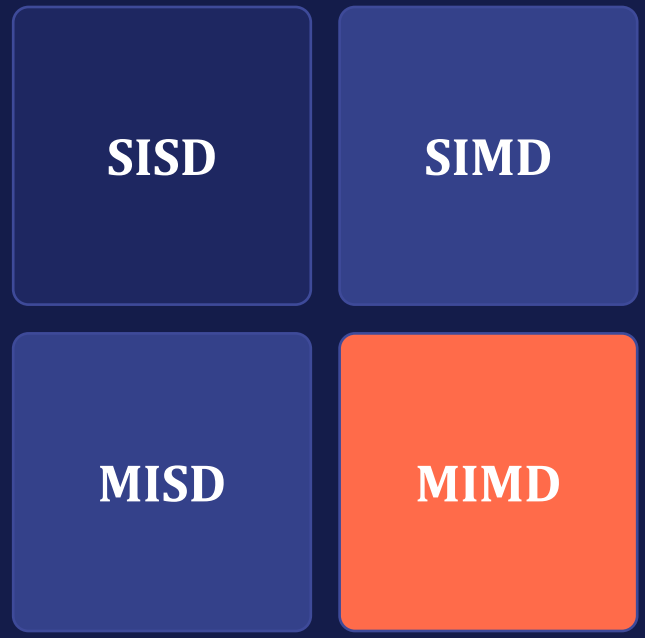
A FRAMEWORK FOR CLASSIFICATION

Flynn's Classification of Computer Architectures

In 1966, Michael Flynn proposed classifying computer architectures by the number of instruction streams and data streams they process — a stream being simply a sequence of items, whether data or instructions.

Key idea

Two independent questions define a machine's class:
1. How many instruction streams does it process?
2. How many data streams does it process?



Single vs. multiple instruction streams (rows) × single vs. multiple data streams (columns)

THE FOUR CLASSES

Flynn's Taxonomy at a Glance

INSTRUCTION STREAMS \ DATA STREAMS	Single	Multiple
Single	**SISD** Classical von Neumann — most PCs, single-CPU workstations	**SIMD** Array & vector processors — GPUs
Multiple	**MISD** Non-existent in practice — listed for completeness	**MIMD** Most common general-purpose parallel machine

FLYNN CLASS 1 OF 4

SISD — Single Instruction, Single Data

The traditional mono-processor (von Neumann) design: one instruction stream operates on one data stream, one instruction per clock cycle. SISD machines are usually pipelined internally even though they process a single stream.

- One control unit issues instructions to exactly one processing element.
- That processing element reads and writes exactly one data stream.
- Instructions execute strictly one at a time, in program order (at the architectural level).

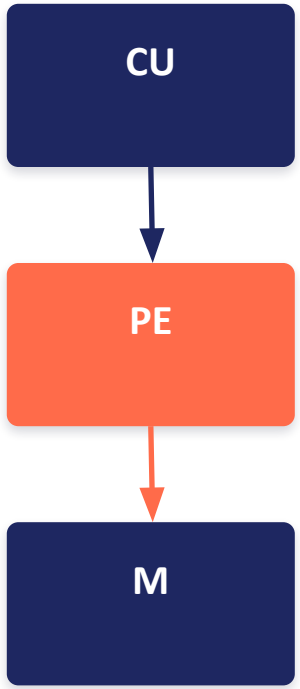
REAL-WORLD EXAMPLE

Everyday single-CPU systems

Most personal computers and single-CPU workstations of the pre-multicore era. A basic calculator or a simple embedded microcontroller running one program is a clean modern example.

SISD — Architecture

CU = Control Unit PE = Processing Element M = Memory



Instruction stream (IS)

Data stream (DS)

One CU issues one instruction stream to one PE, which exchanges one data stream with memory.

FLYNN CLASS 2 OF 4

SIMD — Single Instruction, Multiple Data

A single control unit broadcasts one instruction stream to many processing elements. Every unit executes the same instruction in the same clock cycle, but each operates on a different data element — ideal for array and vector processing.

- One CU, many PEs (or ALUs) executing in lock-step.
- Each PE applies the identical operation to its own slice of data simultaneously.
- This class covers array processors, and vector processors are often grouped here too.

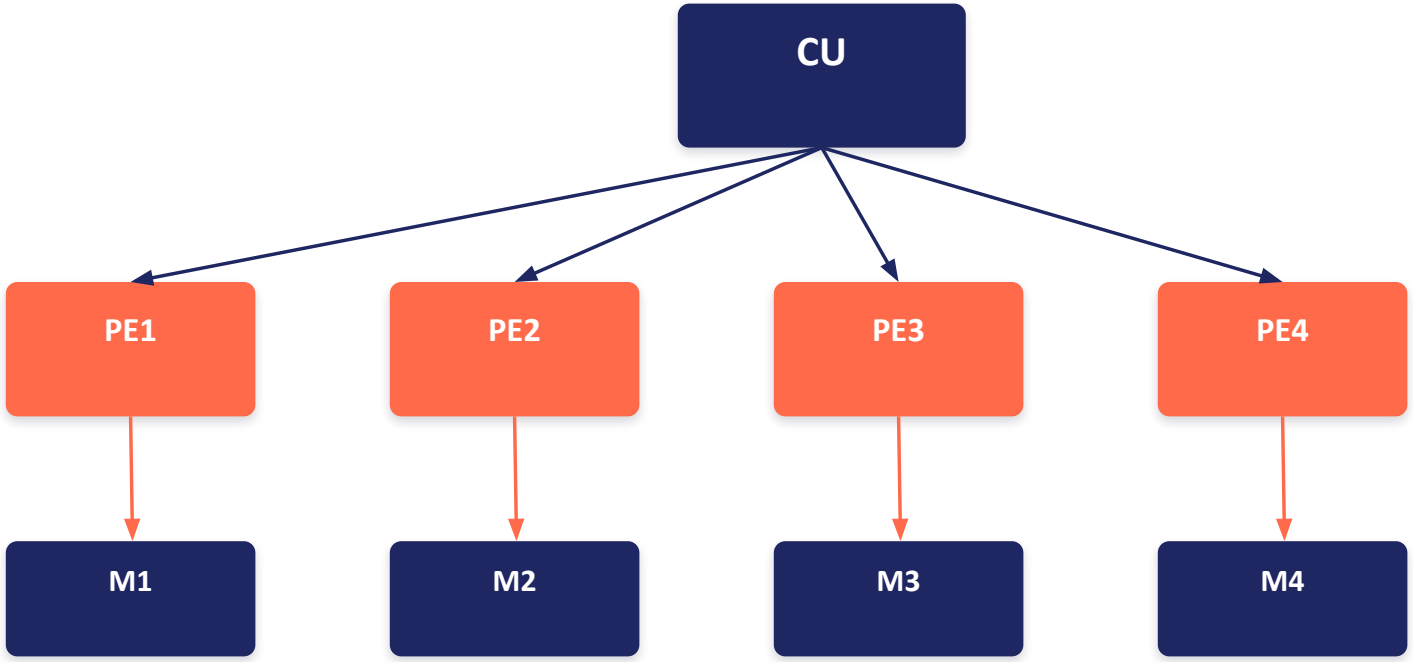
REAL-WORLD EXAMPLE

GPUs and vector instructions

Graphics Processing Units (GPUs) apply the same shader instruction to thousands of pixels at once. On CPUs, Intel AVX-512 vector instructions add 16 floating-point numbers in a single instruction.

SIMD — Architecture

CU = Control Unit PE = Processing Element M = Memory



Every PE executes the same instruction each cycle, but on its own data stream.

FLYNN CLASS 3 OF 4

MISD — Multiple Instructions, Single Data

Multiple processing units each run a different instruction stream, but all of them operate on the very same single data stream. This organization has essentially never been used in practice — it is listed for completeness in Flynn's taxonomy.

- Each processor executes its own, distinct sequence of instructions.
- All processors consume the same one data stream flowing through the system.
- No mainstream commercial computer has been built purely this way.

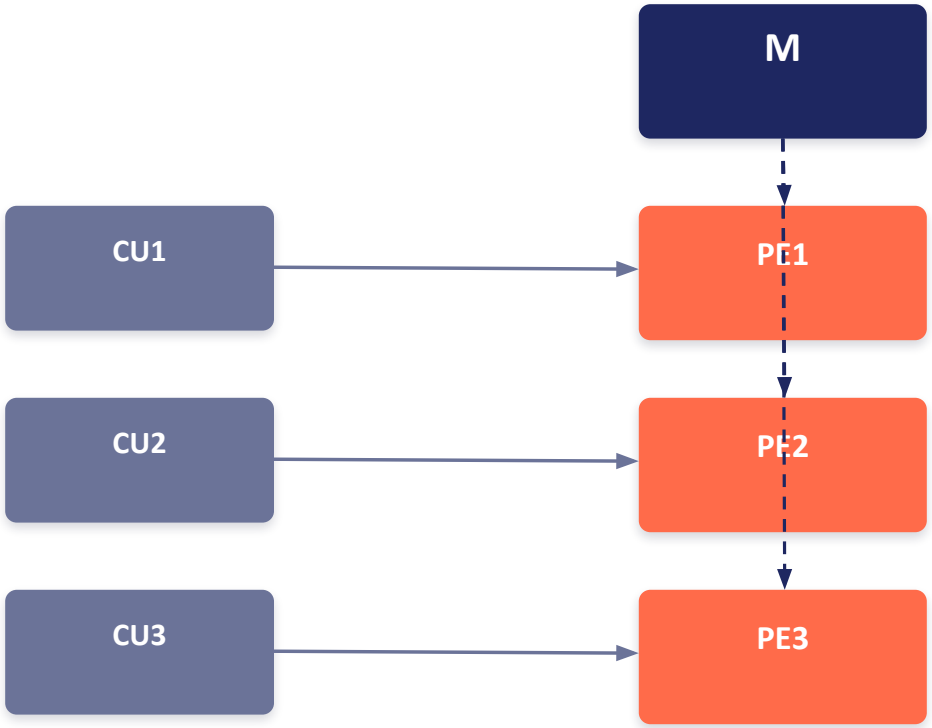
REAL-WORLD EXAMPLE

Closest real analogues

Highly redundant fault-tolerant control systems — e.g., running several different algorithms on the same sensor data stream to cross-check results, as proposed for spacecraft flight-control research — are the closest conceptual match, though rarely called MISD in production systems.

MISD — Architecture

CU = Control Unit PE = Processing Element M = Memory



Multiple CUs each drive a different PE with a different instruction stream — but every PE reads the same single data stream. Rarely implemented in practice.

FLYNN CLASS 4 OF 4

MIMD — Multiple Instructions, Multiple Data

Each processor runs its own separate program, generating its own instruction stream, and operates on its own data. MIMD is the traditional multiprocessor organization, and today the most common and general form of parallel machine.

- Every processing unit has an independent instruction stream and independent data.
- Multiple full programs can execute simultaneously, in parallel.
- Covers everything from multicore desktops to warehouse-scale clusters.

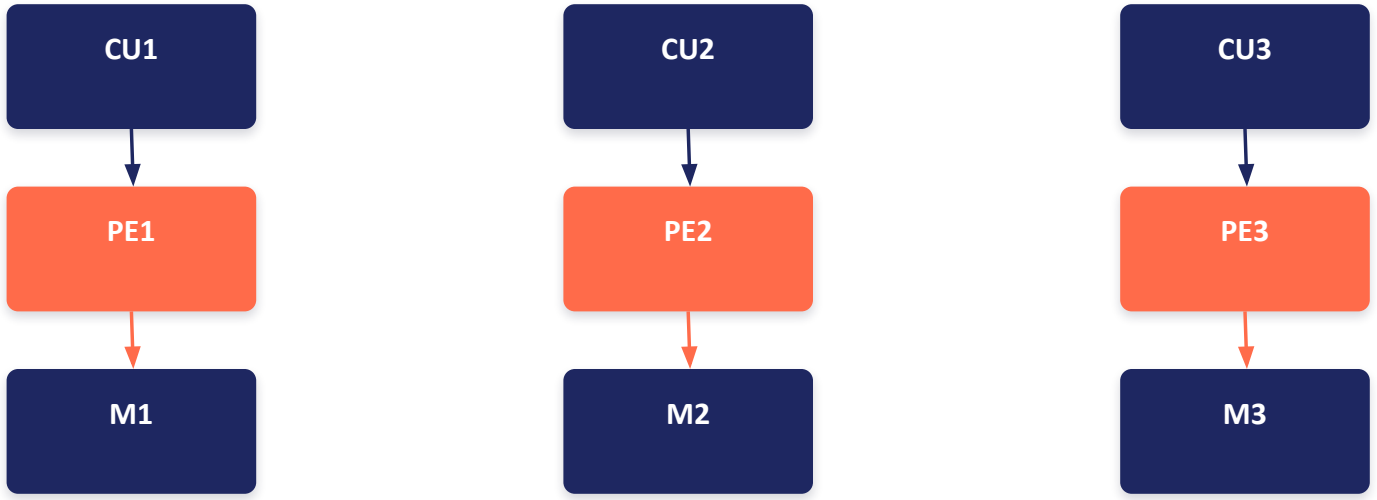
REAL-WORLD EXAMPLE

Multicore CPUs & clusters

A quad-core laptop CPU running a browser, a compiler, and a music player at once; large-scale supercomputers and clusters such as those built for Hadoop/Spark analytics; grid computing systems spanning many machines.

MIMD — Architecture

CU = Control Unit PE = Processing Element M = Memory



Each processing unit has its own instruction stream and its own data stream — fully independent execution, optionally sharing memory.

WRAP-UP

Flynn's Taxonomy — Side by Side

Class	Instruction Streams	Data Streams	Typical Example
SISD	Single	Single	Single-CPU PC / workstation
SIMD	Single	Multiple	GPU, AVX vector units
MISD	Multiple	Single	Rare — redundant fault-tolerant systems
MIMD	Multiple	Multiple	Multicore CPUs, clusters, supercomputers

Takeaway: ILP hides latency within a single instruction stream; DLP and SIMD spread one instruction over many data items; MIMD scales by giving every unit its own full program. Real systems usually combine several of these forms.



Questions?

CS-401 · Parallel and Distributed Computing · Lecture 3

Dr. Tariq Sadad · Department of Computer Science, IIUI